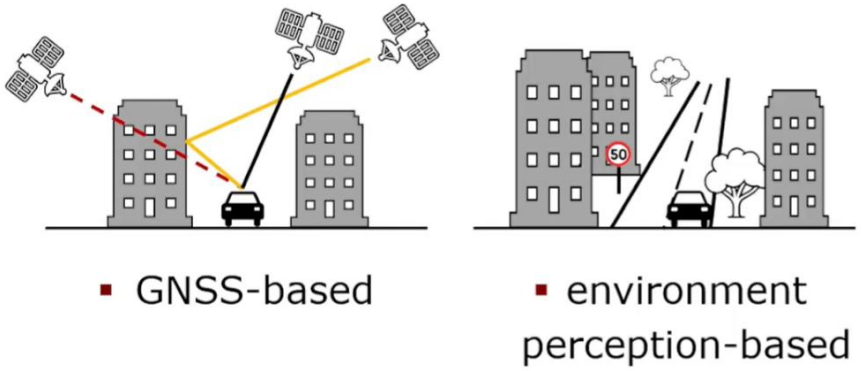
香港城市大學
City University of Hong Kong
Innovating into the Future

Chapter 2: Localization

Localization Techniques

GNSS-based vs Perception-based



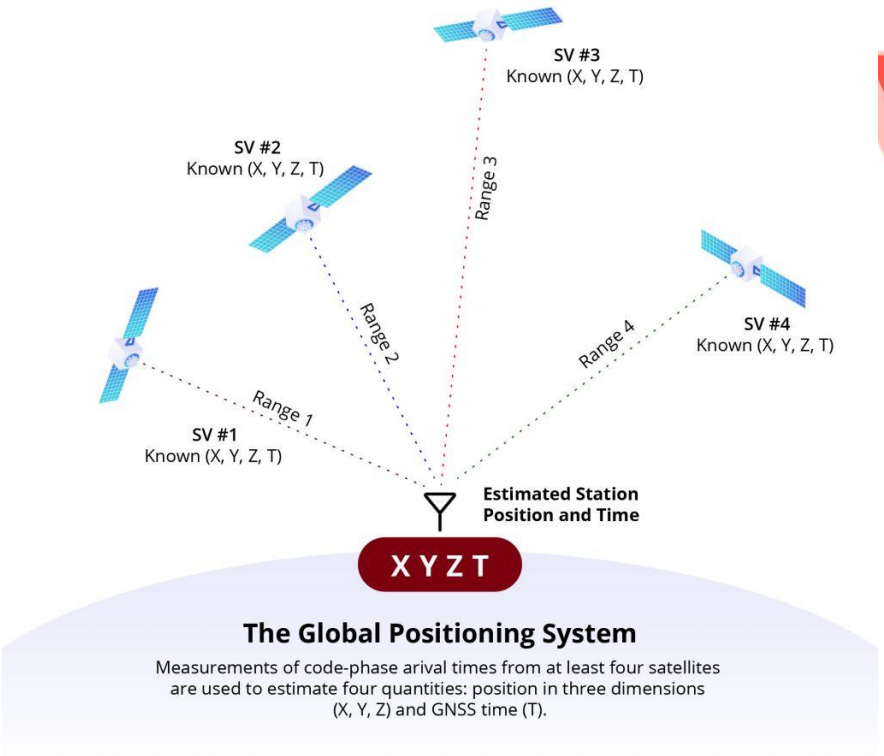
Aspect	GNSS-Based	Environment Perception-Based
Position Type	Absolute (global coordinates)	Relative (to environment) or map-relative absolute
Coverage	Global outdoors, limited indoors	Local, works anywhere with perceivable environment
Accuracy	3-10 meters (standard), cm-level (with correction)	Centimeter-level (relative), meter-level (map-based absolute)
Precision	Low to high (depending on corrections)	Very high (short-term relative motion)
Update Rate	1-10 Hz (typical), limited by satellite geometry	10-100+ Hz (sensor-dependent)
Drift	None (absolute reference)	Significant (odometry-based), none (map-based absolute)
Initialization	Instant (when signals acquired)	Required (needs initial pose or mapping)

Localization Techniques

Aspect	GNSS-Based	Environment Perception-Based
Signal Dependency	Requires clear sky view	Requires perceivable environment (features/landmarks)
Environmental Impact	Affected by weather, multipath	Affected by lighting, weather, seasonal changes
Computational Load	Low (signal processing)	High (sensor data processing, feature extraction, matching)
Infrastructure Cost	None (satellites already deployed)	Variable (sensors onboard, may need external infrastructure)
Power Consumption	Low	High (especially LiDAR, high-res cameras)
Security	Vulnerable to spoofing, jamming	Vulnerable to environment manipulation, sensor attacks
Scalability	Infinite (same for all users)	Limited by map maintenance, processing resources
Orientation Output	Limited (needs multiple antennas)	Full 6-DOF (position + orientation)
Context Awareness	None (just position)	High (understands environment, objects, semantics)
Primary Sensors	GNSS receivers	Cameras, LiDAR, radar, IMU, ultrasonic
Best For	Large-scale navigation, outdoor positioning, global reference	Precise local navigation, GNSS-denied areas, obstacle avoidance
Failure Modes	Urban canyons, indoors, tunnels, jamming	Featureless environments, dynamic changes, sensor failure

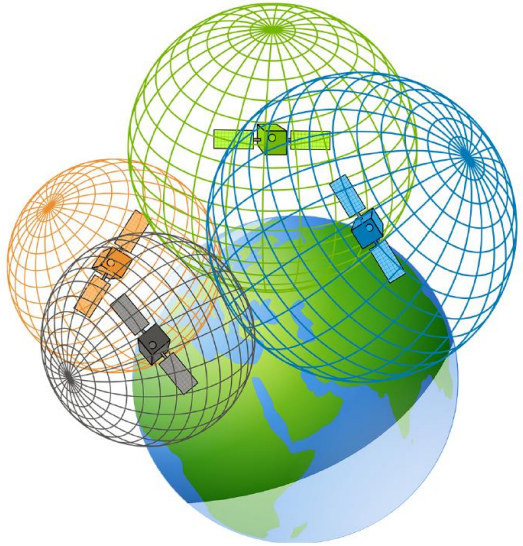
GNSS-based Localization

- **GNSS (**Global Navigation Satellite System) provides global positioning by measuring satellite signal travel time, including GPS, Galileo, BeiDou, GLONASS …
- With corrections, it delivers 1–2 cm absolute accuracy in open-sky conditions
- Long-term stability and true global coordinates
- It suffers from signal blockage and multipath in urban canyons, tunnels etc.
- Tight fusion with IMU, LiDAR, and cameras is necessary for robust all-environment localization



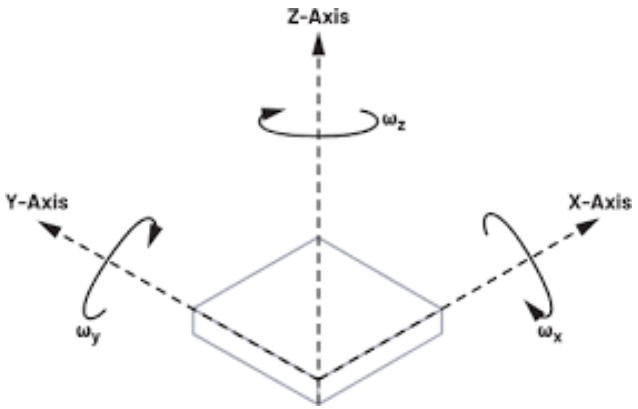
GNSS-based Localization

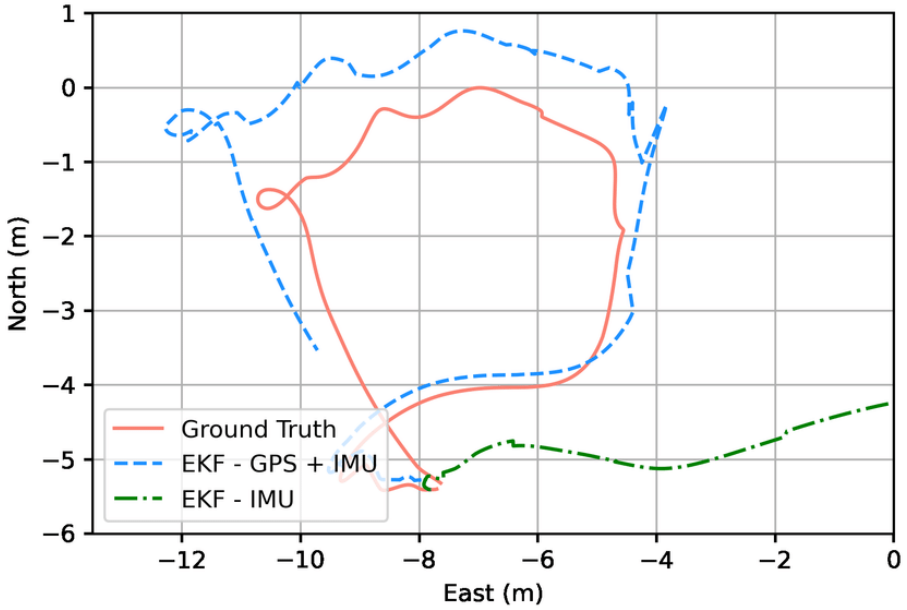
- Determine location by measuring distances to satellites at known locations using the geometry of circles and spheres
- Distance to each satellite is measured as time delay between sent signal/code and received signal/code
- Satellites equipped with onboard atomic clock
- Given 1 satellite, we know which sphere we are on
- Given 2 satellites, we know which circle we are on
- Given 3 satellites, we know the location up to 2 possibilities
- Given 4 satellites, we know the exact location



GNSS-based Localization

- **IMU** (Inertial Measurement Unit) is a sensor that measures and reports an object's linear acceleration and angular rate of rotation
- Typically using a combination of accelerometers and gyroscopes, which can be used to calculate the kinematic states and current location of the vehicle

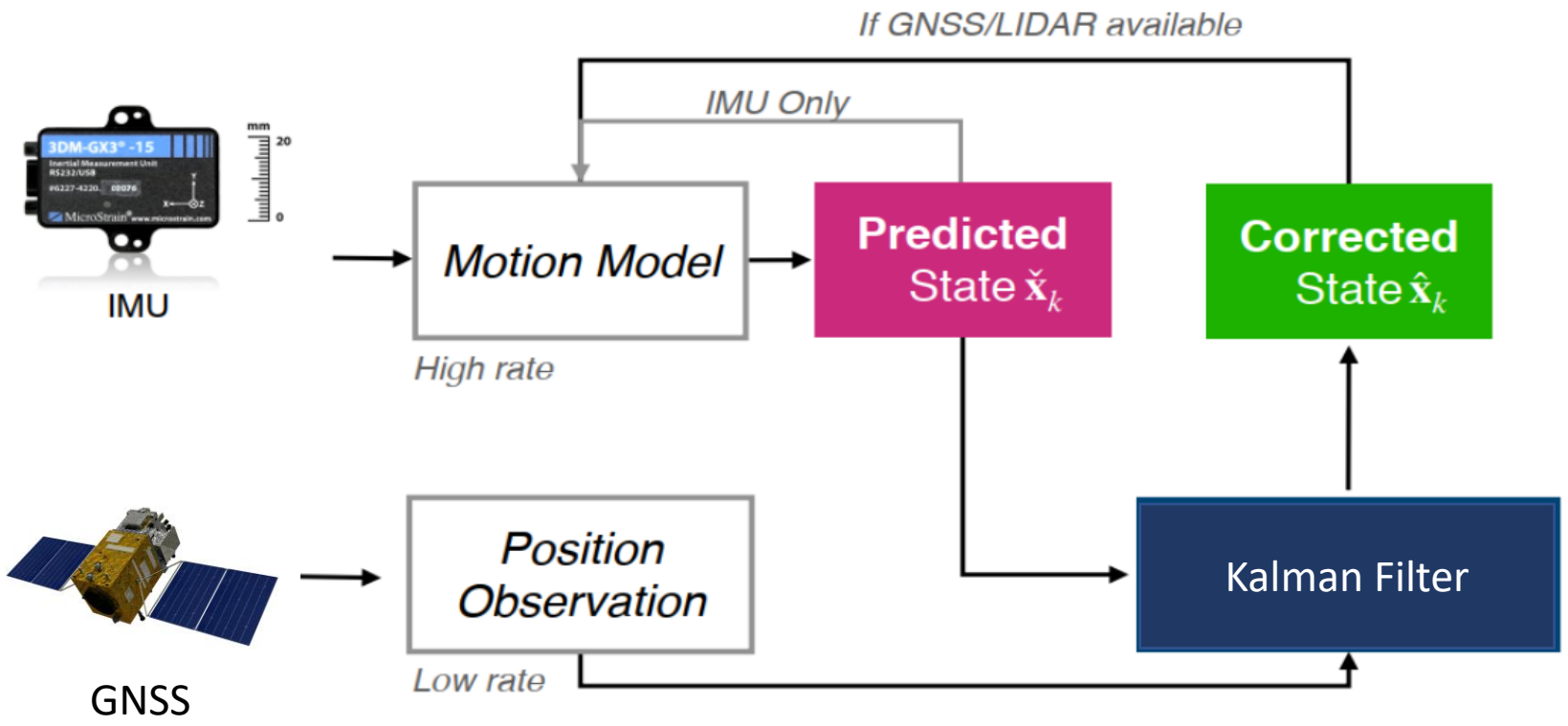




IMU vs GNSS

	IMU	GNSS
Principle	Measures internal motion: acceleration (accelerometer) and rotational rate (gyroscope). Calculates position via dead reckoning.	Measures external signals from satellites. Uses triangulation/timing to determine absolute position.
Output	Relative position, velocity, and orientation change from a known starting point.	Absolute global coordinates (latitude, longitude, altitude).
Accuracy (Short Term)	Very high for short periods/distance. No short-term drift.	Moderate to High (1-5 meters civilian, cm-level with correction).
Long-Term Drift	High (Critical Weakness). Errors (especially from bias & noise) integrate over time, causing unbounded position drift.	Low/None. No drift over time. Accuracy is independent of time.
Environment Dependency	Independent. Works everywhere (indoors, underwater, tunnels).	Dependent. Requires clear line-of-sight to satellites. Fails indoors, underwater, urban canyons.
Update Rate	Very High (100 Hz - 1 kHz). Ideal for high-speed control.	Low (1-10 Hz typical). Not suitable for very high-speed control loops.
Initialization	Requires initial position and orientation from an external source (e.g., GPS).	Provides initialization instantly (time to first fix).
Primary Error Source	Sensor biases, noise, scale factor errors	Atmospheric delays, multipath, satellite geometry, signal blockage.
Cost	Varies widely. Consumer-grade ($10) drifts in seconds. Tactical/Industrial ($1k-$10k) drifts slower. Aviation/Grade ($100k+) for minimal drift.	Low. Consumer chipsets are very cheap. High-precision RTK systems are more expensive ($1k-$10k).

Combining IMU and GNSS

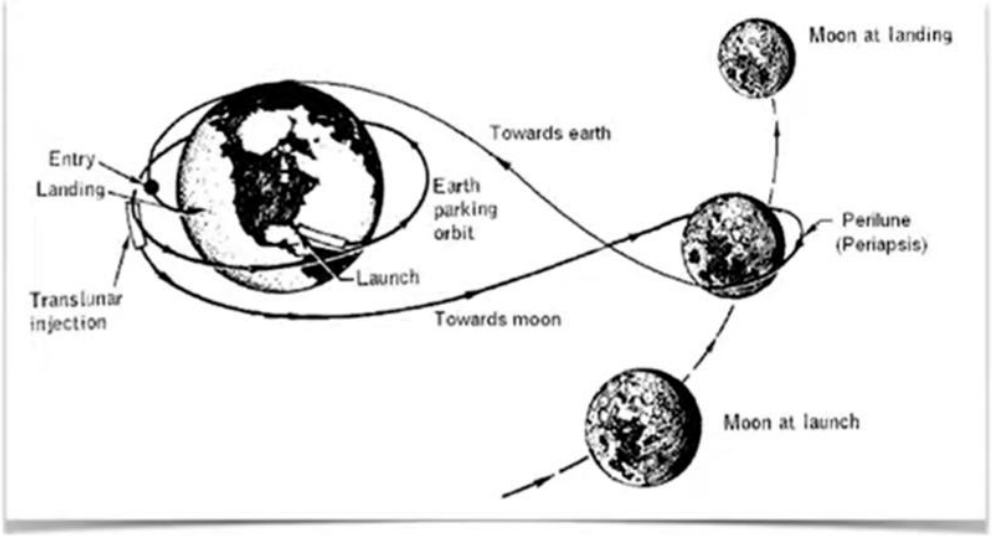


Kalman Filter

- A **recursive Bayesian estimation algorithm** designed to estimate the state of a dynamic system from noisy measurements
- It optimally fuses prior state information and current observations to produce a more accurate state estimate
- Assumptions:
  - System is *linear* (state transition & measurement models follow linear equations)
    (can be extended to non-linear systems)
  - Noise (process noise & measurement noise) is *Gaussian distributed*
    (zero mean, known covariance matrices)

Kalman Filter

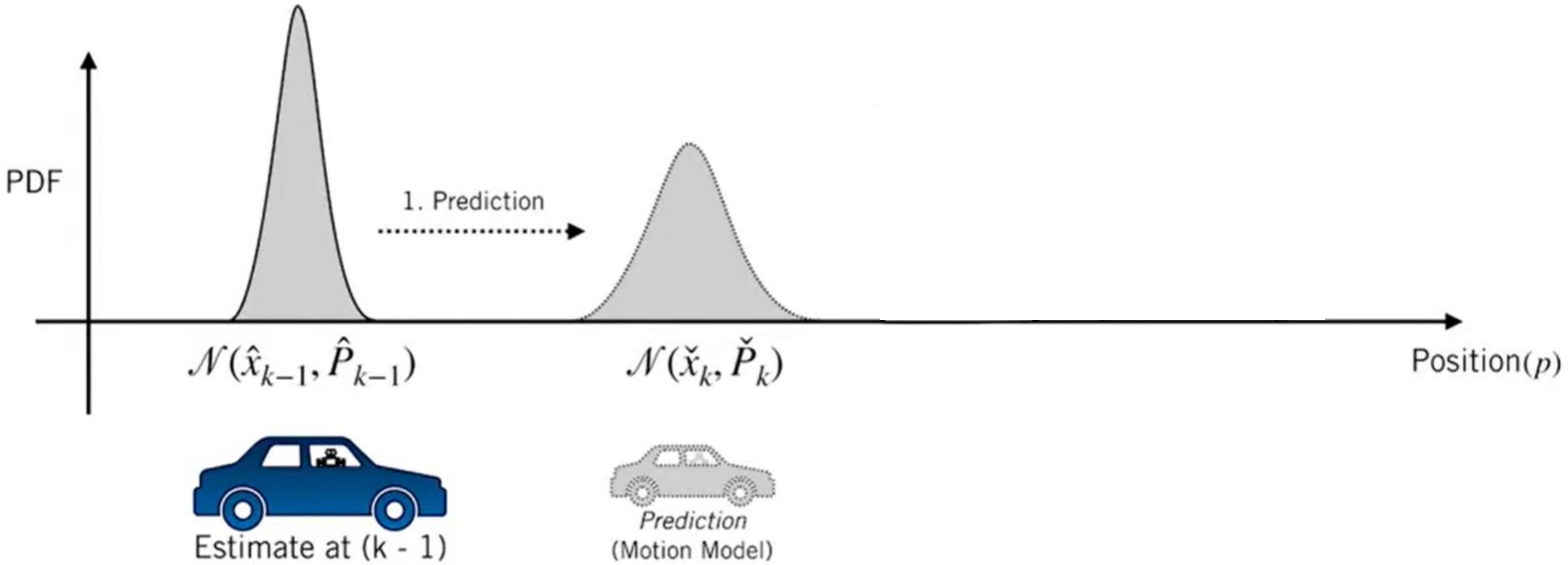
Apollo Guidance Computer



The (extended) Kalman Filter became widely known after its use in the Apollo Guidance Computer for circumlunar navigation.

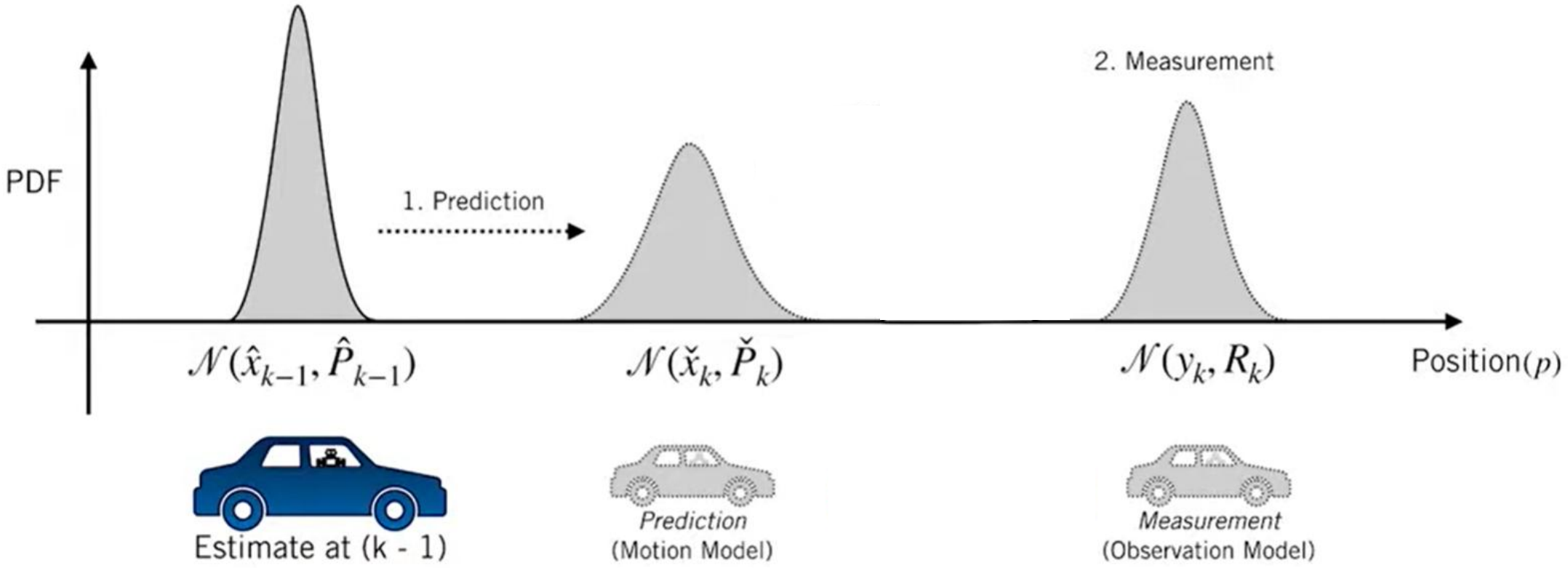
Kalman Filter

Kalman Filter



IMU

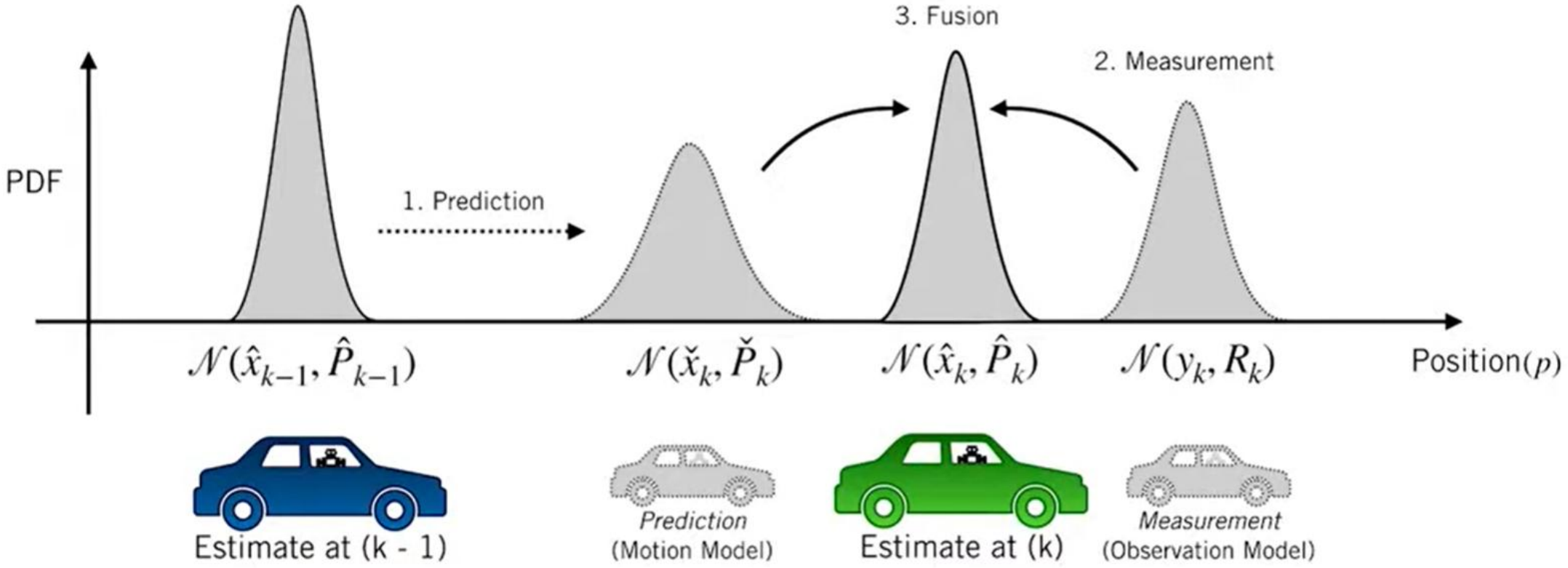
Kalman Filter



IMU

GPS

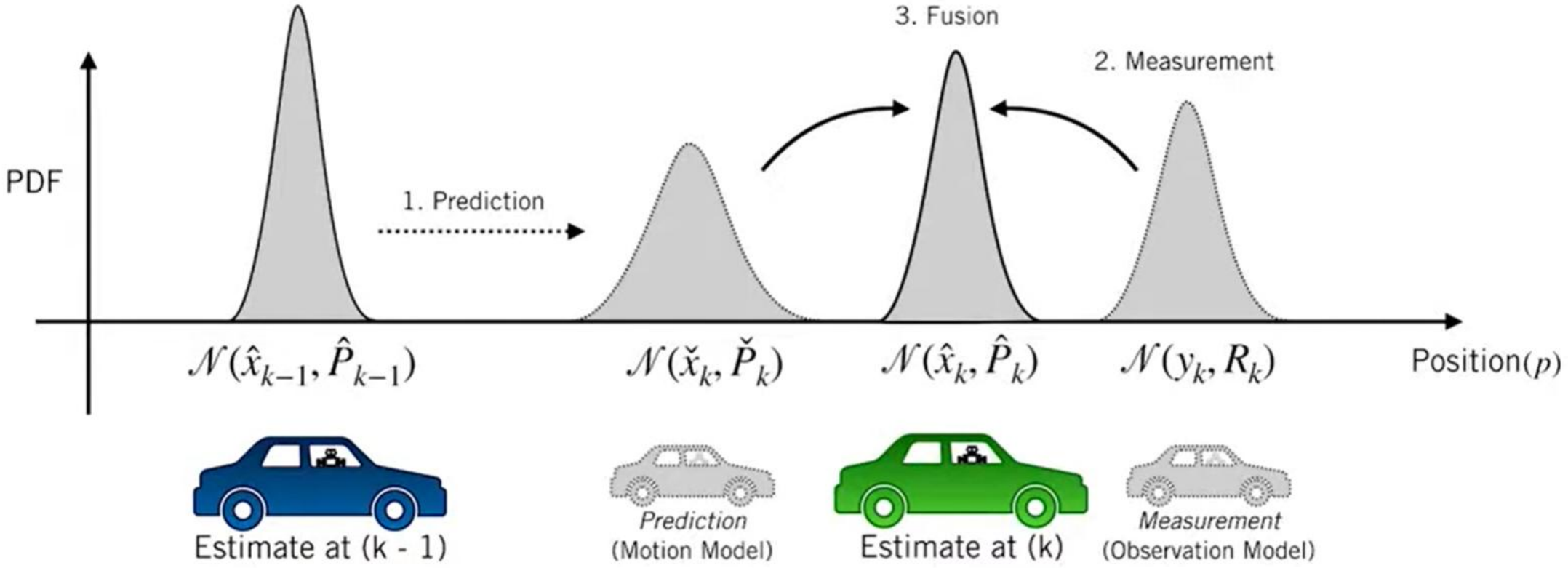
Kalman Filter



IMU

GPS

Kalman Filter



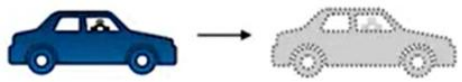
IMU

GPS

Kalman Filter

- The Kalman Filter requires the following motion and measurement models

Motion model: $$\mathbf{x}_k = \mathbf{F}_{k-1}\mathbf{x}_{k-1} + \underbrace{\mathbf{G}_{k-1}\mathbf{u}_{k-1}}_{input} + \underbrace{\mathbf{w}_{k-1}}_{noise}$$



Measurement model: $$\mathbf{y}_k = \mathbf{H}_k\mathbf{x}_k + \underbrace{\mathbf{v}_k}_{noise}$$

- With the following noise properties

$$\mathbf{v}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{R}_k)$$

Measurement Noise

$$\mathbf{w}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}_k)$$

Process or Motion Noise

Kalman Filter

1 Prediction

$$\check{\mathbf{x}}_k = \mathbf{F}_{k-1}\mathbf{x}_{k-1} + \mathbf{G}_{k-1}\mathbf{u}_{k-1}$$

$$\check{\mathbf{P}}_k = \mathbf{F}_{k-1}\hat{\mathbf{P}}_{k-1}\mathbf{F}_{k-1}^T + \mathbf{Q}_{k-1}$$

2b Correction

$$\hat{\mathbf{x}}_k = \check{\mathbf{x}}_k + \mathbf{K}_k(\mathbf{y}_k - \mathbf{H}_k\check{\mathbf{x}}_k)$$

$$\hat{\mathbf{P}}_k = (\mathbf{1} - \mathbf{K}_k\mathbf{H}_k)\check{\mathbf{P}}_k$$

$$(\mathbf{y}_k - \mathbf{H}_k\check{\mathbf{x}}_k)$$

is often called the 'innovation'

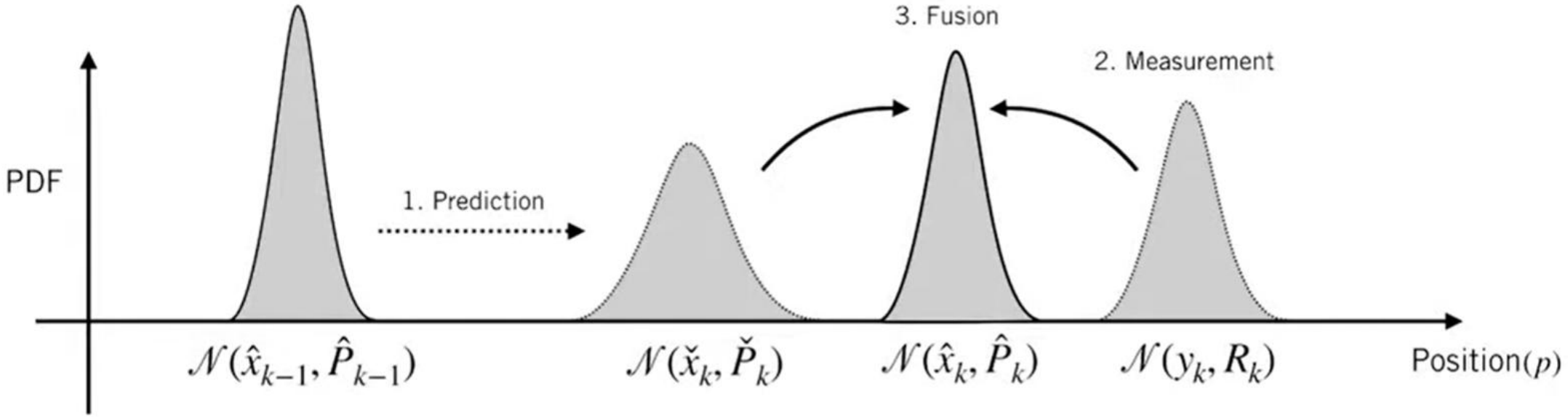
2a Optimal Gain

$$\mathbf{K}_k = \check{\mathbf{P}}_k\mathbf{H}_k^T(\mathbf{H}_k\check{\mathbf{P}}_k\mathbf{H}_k^T + \mathbf{R}_k)^{-1}$$

Prediction
$\check{\mathbf{x}}_k$ *(given motion model)*
at time k

Corrected prediction
$\hat{\mathbf{x}}_k$ (given measurement)
at time k

Kalman Filter



prediction

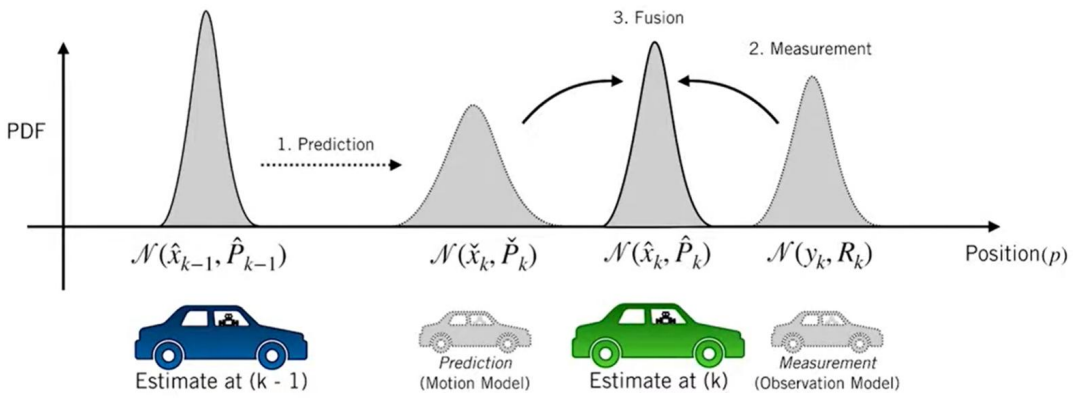
$$\check{\mathbf{x}}_k = \mathbf{F}_{k-1}\mathbf{x}_{k-1} + \mathbf{G}_{k-1}\mathbf{u}_{k-1}$$
$$\check{\mathbf{P}}_k = \mathbf{F}_{k-1}\hat{\mathbf{P}}_{k-1}\mathbf{F}_{k-1}^T + \mathbf{Q}_{k-1}$$

measurement & correction

$$\mathbf{K}_k = \check{\mathbf{P}}_k\mathbf{H}_k^T(\mathbf{H}_k\check{\mathbf{P}}_k\mathbf{H}_k^T + \mathbf{R}_k)^{-1}$$
$$\hat{\mathbf{x}}_k = \check{\mathbf{x}}_k + \mathbf{K}_k(\mathbf{y}_k - \mathbf{H}_k\check{\mathbf{x}}_k)$$
$$\hat{\mathbf{P}}_k = (\mathbf{1} - \mathbf{K}_k\mathbf{H}_k)\check{\mathbf{P}}_k$$

Kalman Filter

- Example:



$$\mathbf{x} = \begin{bmatrix} p \\ \frac{dp}{dt} = \dot{p} \end{bmatrix} \qquad \mathbf{u} = a = \frac{d^2p}{dt^2}$$

Motion/Process Model

$$\mathbf{x}_k = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} \mathbf{x}_{k-1} + \begin{bmatrix} 0 \\ \Delta t \end{bmatrix} \mathbf{u}_{k-1} + \mathbf{w}_{k-1}$$

Position Observation

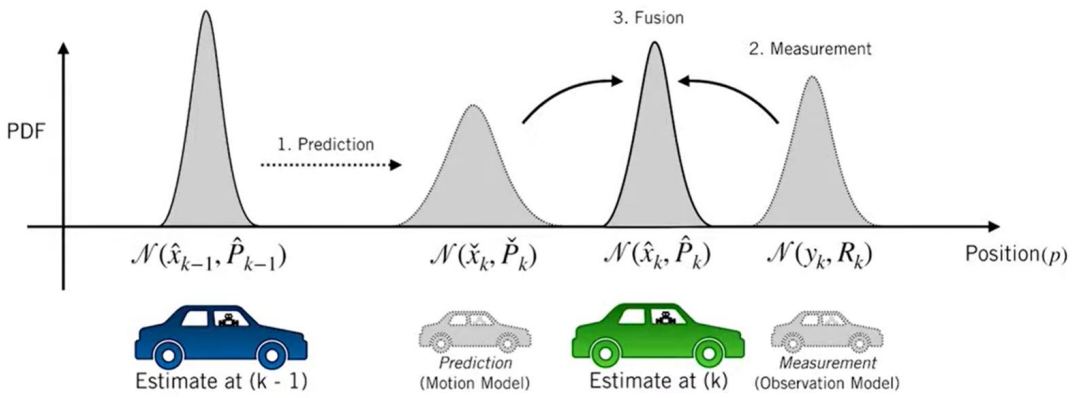
$$y_k = [1 \quad 0]\, \mathbf{x}_k + v_k$$

Noise Densities

$$v_k \sim \mathcal{N}(0,\, 0.05) \qquad \mathbf{w}_k \sim \mathcal{N}(\mathbf{0}, (0.1)\mathbf{1}_{2\times 2})$$

Kalman Filter

- Example:



Data

$$\hat{\mathbf{x}}_0 \sim \mathcal{N}\left(\begin{bmatrix}0\\5\end{bmatrix}, \begin{bmatrix}0.01 & 0\\0 & 1\end{bmatrix}\right)$$

$$\Delta t = 0.5\text{s}$$

$$u_0 = -2\ [m/s^2] \qquad y_1 = 2.2\ [m]$$

Prediction

$$\check{\mathbf{x}}_k = \mathbf{F}_{k-1}\mathbf{x}_{k-1} + \mathbf{G}_{k-1}\mathbf{u}_{k-1}$$

$$\begin{bmatrix}\check{p}_1\\ \check{\dot{p}}_1\end{bmatrix} = \begin{bmatrix}1 & 0.5\\0 & 1\end{bmatrix}\begin{bmatrix}0\\5\end{bmatrix} + \begin{bmatrix}0\\0.5\end{bmatrix}(-2) = \begin{bmatrix}2.5\\4\end{bmatrix}$$

$$\check{\mathbf{P}}_k = \mathbf{F}_{k-1}\hat{\mathbf{P}}_{k-1}\mathbf{F}_{k-1}^T + \mathbf{Q}_{k-1}$$

$$\check{\mathbf{P}}_1 = \begin{bmatrix}1 & 0.5\\0 & 1\end{bmatrix}\begin{bmatrix}0.01 & 0\\0 & 1\end{bmatrix}\begin{bmatrix}1 & 0.5\\0 & 1\end{bmatrix}^T + \begin{bmatrix}0.1 & 0\\0 & 0.1\end{bmatrix} = \begin{bmatrix}0.36 & 0.5\\0.5 & 1.1\end{bmatrix}$$

Correction

$$\mathbf{K}_1 = \check{\mathbf{P}}_1\mathbf{H}_1^T(\mathbf{H}_1\check{\mathbf{P}}_1\mathbf{H}_1^T + \mathbf{R}_1)^{-1}$$

$$= \begin{bmatrix}0.36 & 0.5\\0.5 & 1.1\end{bmatrix}\begin{bmatrix}1\\0\end{bmatrix}\left([1\quad 0]\begin{bmatrix}0.36 & 0.5\\0.5 & 1.1\end{bmatrix}\begin{bmatrix}1\\0\end{bmatrix} + 0.05\right)^{-1}$$

$$= \begin{bmatrix}0.88\\1.22\end{bmatrix}$$

$$\hat{\mathbf{x}}_1 = \check{\mathbf{x}}_1 + \mathbf{K}_1(\mathbf{y}_1 - \mathbf{H}_1\check{\mathbf{x}}_1)$$

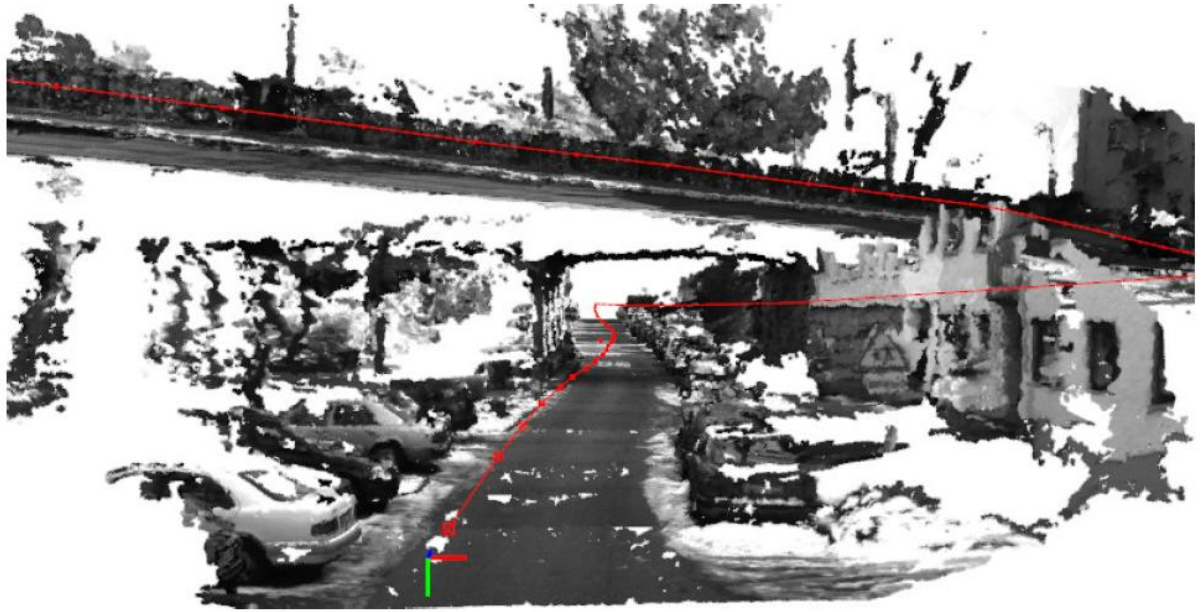
$$\begin{bmatrix}\hat{p}_1\\ \hat{\dot{p}}_1\end{bmatrix} = \begin{bmatrix}2.5\\4\end{bmatrix} + \begin{bmatrix}0.88\\1.22\end{bmatrix}\left(2.2 - [1\quad 0]\begin{bmatrix}2.5\\4\end{bmatrix}\right) = \begin{bmatrix}2.24\\3.63\end{bmatrix}$$

$$\hat{\mathbf{P}}_1 = (\mathbf{1} - \mathbf{K}_1\mathbf{H}_1)\check{\mathbf{P}}_1$$

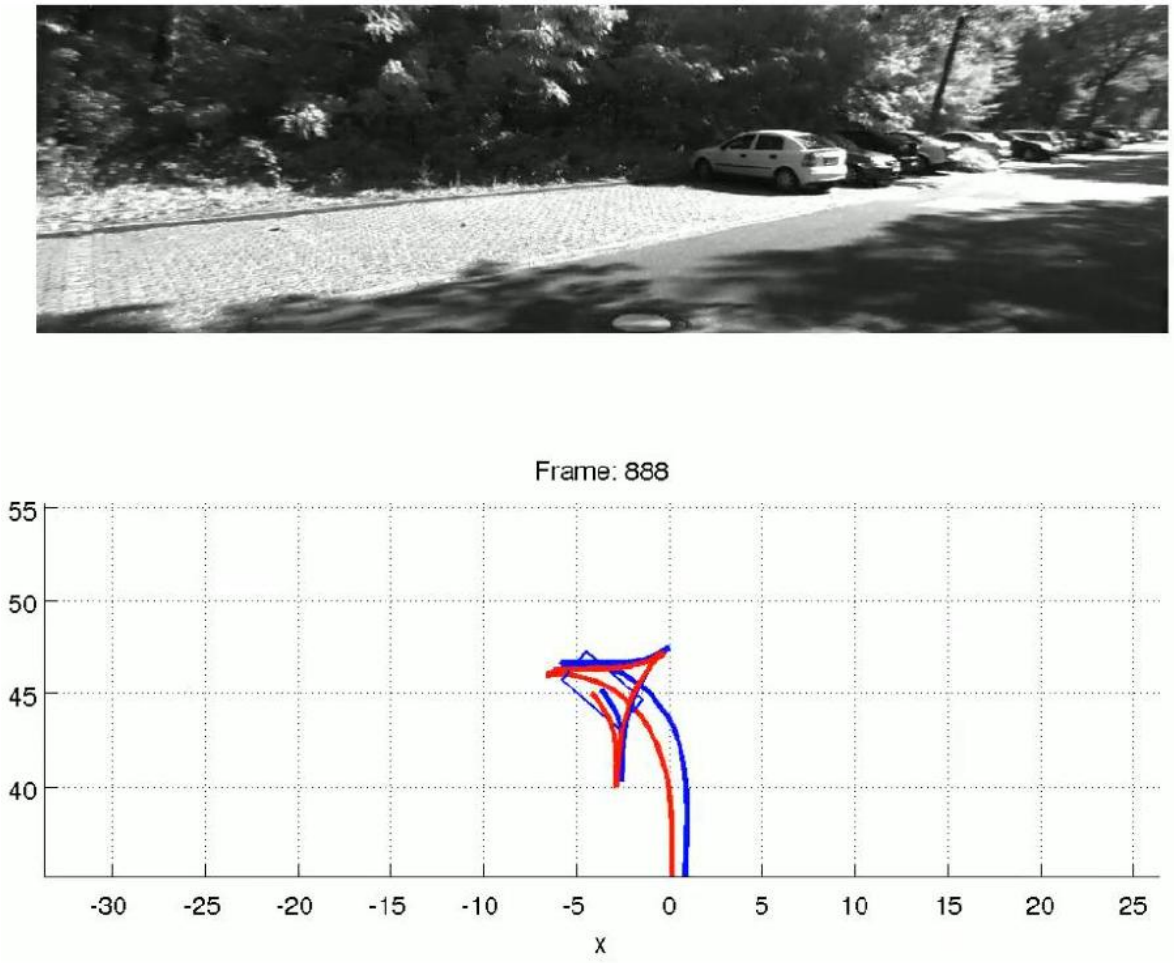
$$= \begin{bmatrix}0.04 & 0.06\\0.06 & 0.49\end{bmatrix}$$

Perception-based Localization

- Tracking the position and orientation of the camera with respect to the environment from its images
- Considers a limited set of recent images
- A (sparse) local map is often built as a by-product, but mapping is not the focus



Perception-based Localization



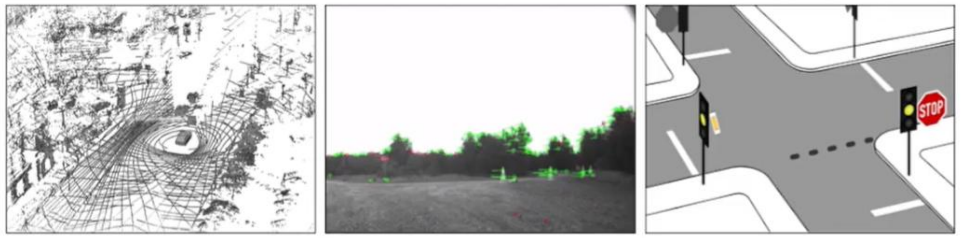
Kitt, Geiger and Lategahn: Visual odometry based on stereo image sequences with RANSAC-based outlier rejection scheme. IV, 2010.

Perception-based Localization: Challenges

- Extreme environmental variability
  - Dynamic/ unstructured settings cause appearance changes and occlusions, disrupting feature matching and distorting perceptual data
- Perceptual aliasing and ambiguous features
  - Repetitive textures, textureless regions, and object shape symmetries create scarce and ambiguous visual features, leading to false-positive matching, cumulative localization drift
- Inherent sensor limitations
  - Camera sensitivity to glare and lack of native depth, LiDAR point cloud sparsity at long ranges, and imprecise spatiotemporal calibration for multi-sensor fusion all amplify positioning errors
- Accuracy-real-time performance tradeoff:
  - State-of-the-art perception algorithms demand heavy computational resources, which conflicts with the resource constraints of embedded platforms used in most autonomous systems

Raw Data vs. Features vs. Landmarks

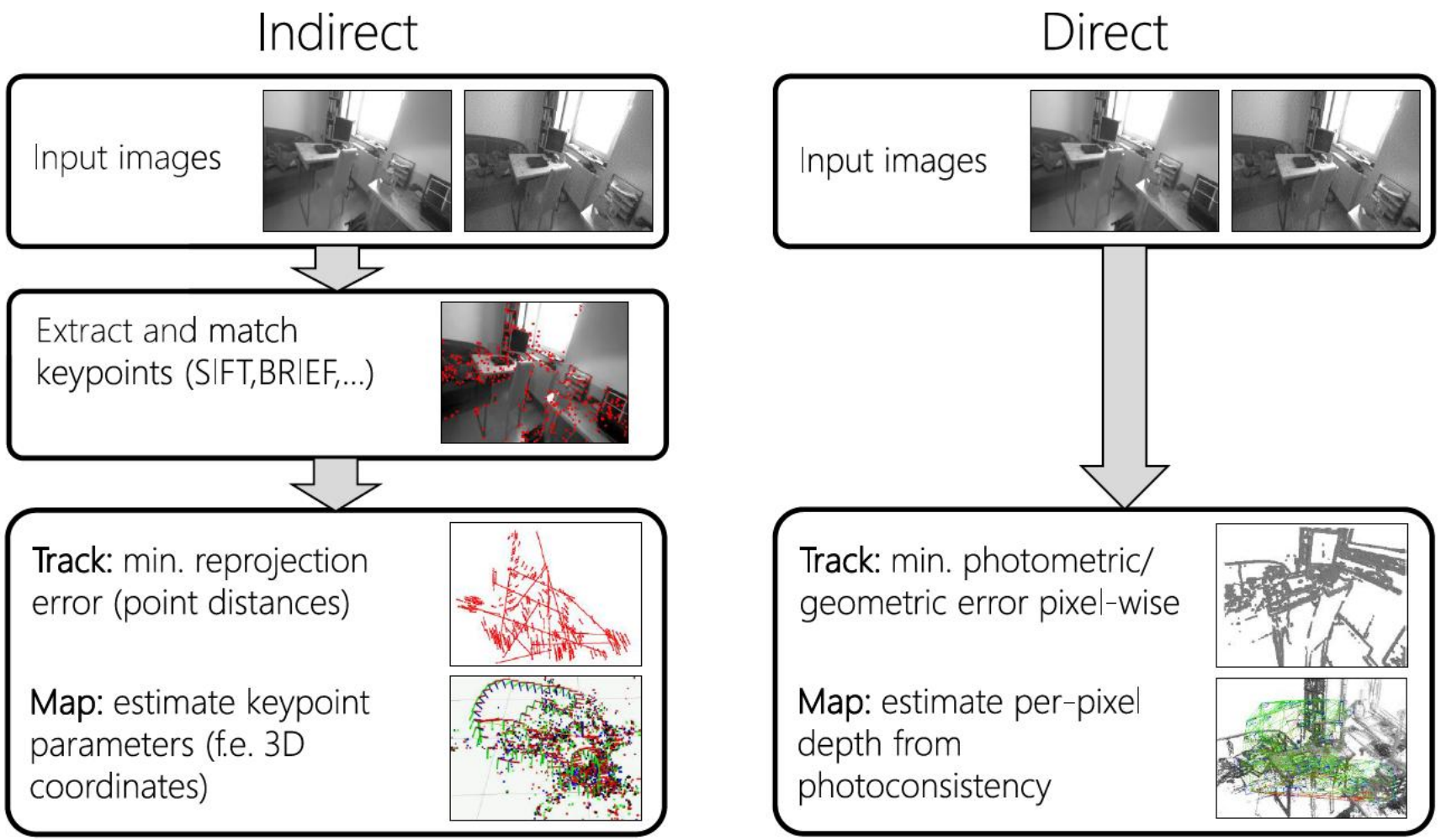
Three patterns:



Raw data Features Landmarks

Pattern	Advantages	Disadvantages
Raw Data	1. Least abstraction level 2. Preserves all environmental information 3. Functions in texture-poor settings	1. High computational intensity 2. Prone to drift 3. Highly sensitive to noise and environmental changes
Feature	1. Balances efficiency and robustness well 2. Reduces data volume 3. Enables reliable relative positioning	1. Fails in low-texture or repetitive environments 2. Suffers from incremental drift
Landmark	1. Highest abstraction level 2. Provides highly accurate, absolute positioning 3. Low computational overhead 4. Robust to changes between landmarks	1. Requires pre-existing map or deployed beacon system 2. Vulnerable to landmark occlusion or alteration

Indirect vs. Direct Methods



Extract and Match Keypoints

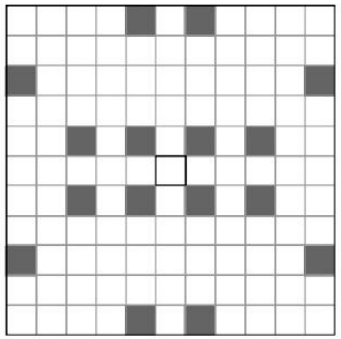
- Detect **salient points** in the image (blobs, corners) and extract local features
- Features should be invariant to perspective and illumination changes
- Match features between two images by their similarity correspondences

-1	-1	-1	-1	-1
-1	+1	+1	+1	-1
-1	+1	+8	+1	-1
-1	+1	+1	+1	-1
-1	-1	-1	-1	-1

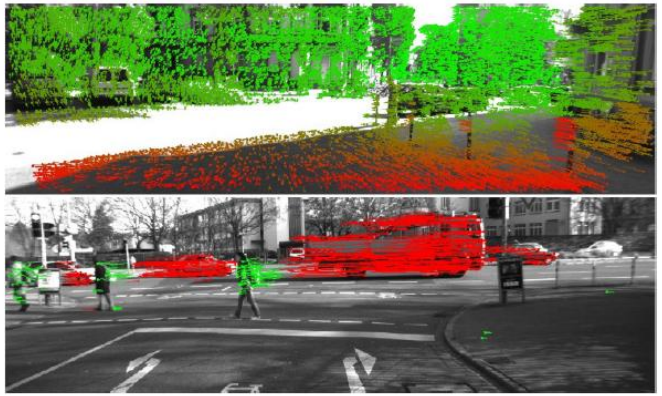
(a) Blob detector

-1	-1	0	+1	+1
-1	-1	0	+1	+1
0	0	0	0	0
+1	+1	0	-1	-1
+1	+1	0	-1	-1

(b) Corner detector



(c) Feature descriptor



2D Points

- 2D points can be written in **inhomogeneous coordinates** as

$$\mathbf{x} = \begin{pmatrix} x \\ y \end{pmatrix} \in \mathbb{R}^2$$

- or in **homogeneous coordinates** as

$$\tilde{\mathbf{x}} = \begin{pmatrix} \tilde{x} \\ \tilde{y} \\ \tilde{w} \end{pmatrix} \in \mathbb{P}^2$$

2D Points

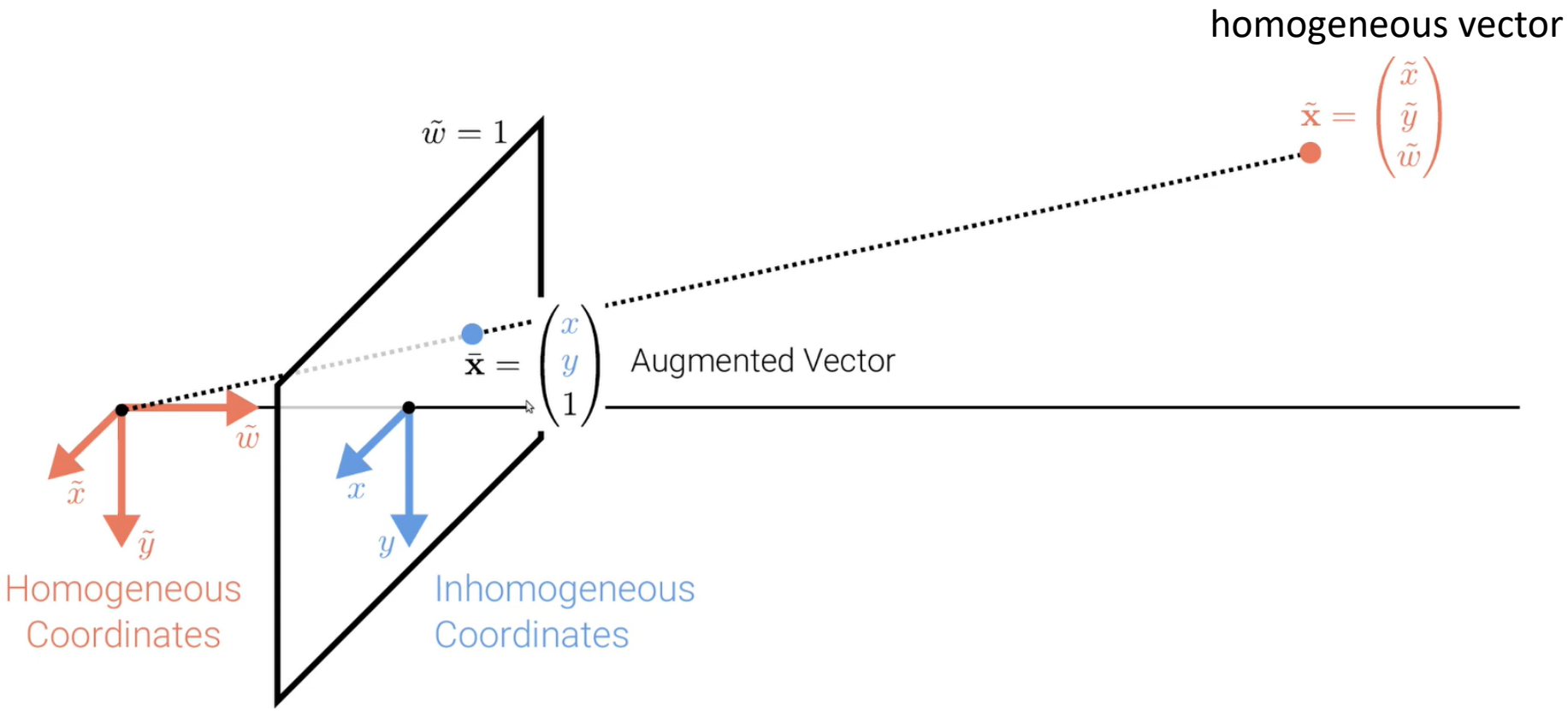
- An **inhomogeneous vector** $\mathbf{x}$ is converted to a **homogeneous vector** $\tilde{\mathbf{x}}$:

$$\tilde{\mathbf{x}} = \begin{pmatrix} \tilde{x} \\ \tilde{y} \\ \tilde{w} \end{pmatrix} = \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} \mathbf{x} \\ 1 \end{pmatrix} = \bar{\mathbf{x}}$$

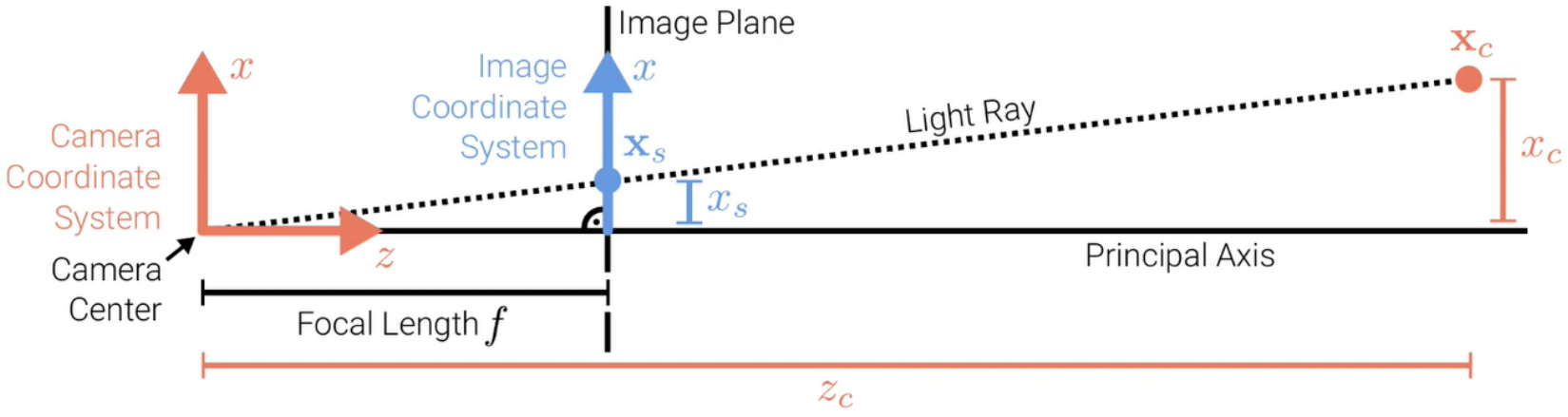
- with **augmented vector** $\bar{\mathbf{x}}$. To convert in the opposite direction we divide by $\tilde{w}$:

$$\bar{\mathbf{x}} = \begin{pmatrix} \mathbf{x} \\ 1 \end{pmatrix} = \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \frac{1}{\tilde{w}} \tilde{\mathbf{x}} = \frac{1}{\tilde{w}} \begin{pmatrix} \tilde{x} \\ \tilde{y} \\ \tilde{w} \end{pmatrix} = \begin{pmatrix} \tilde{x}/\tilde{w} \\ \tilde{y}/\tilde{w} \\ 1 \end{pmatrix}$$

2D Points


homogeneous vector
$\tilde{\mathbf{x}} = \begin{pmatrix} \tilde{x} \\ \tilde{y} \\ \tilde{w} \end{pmatrix}$
$\tilde{w} = 1$
$\bar{\mathbf{x}} = \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$
Augmented Vector
$\tilde{w}$
$\tilde{x}$
$\tilde{y}$
x
y
Homogeneous Coordinates
Inhomogeneous Coordinates

Perspective Projection



$$\frac{x_s}{f} = \frac{x_c}{z_c}$$

Perspective projection of a 3D point $\mathbf{x}_c \in \mathbb{R}^3$ to pixel coordinates $\mathbf{x}_s \in \mathbb{R}^2$:

- The light ray passes through the camera center, the pixel $\mathbf{x}_s$ and the point $\mathbf{x}_c$
- Convention: the principal axis (orthogonal to image plane) aligns with the z-axis
- Remark: the y coordinate is not shown here for clarity, but behaves similarly

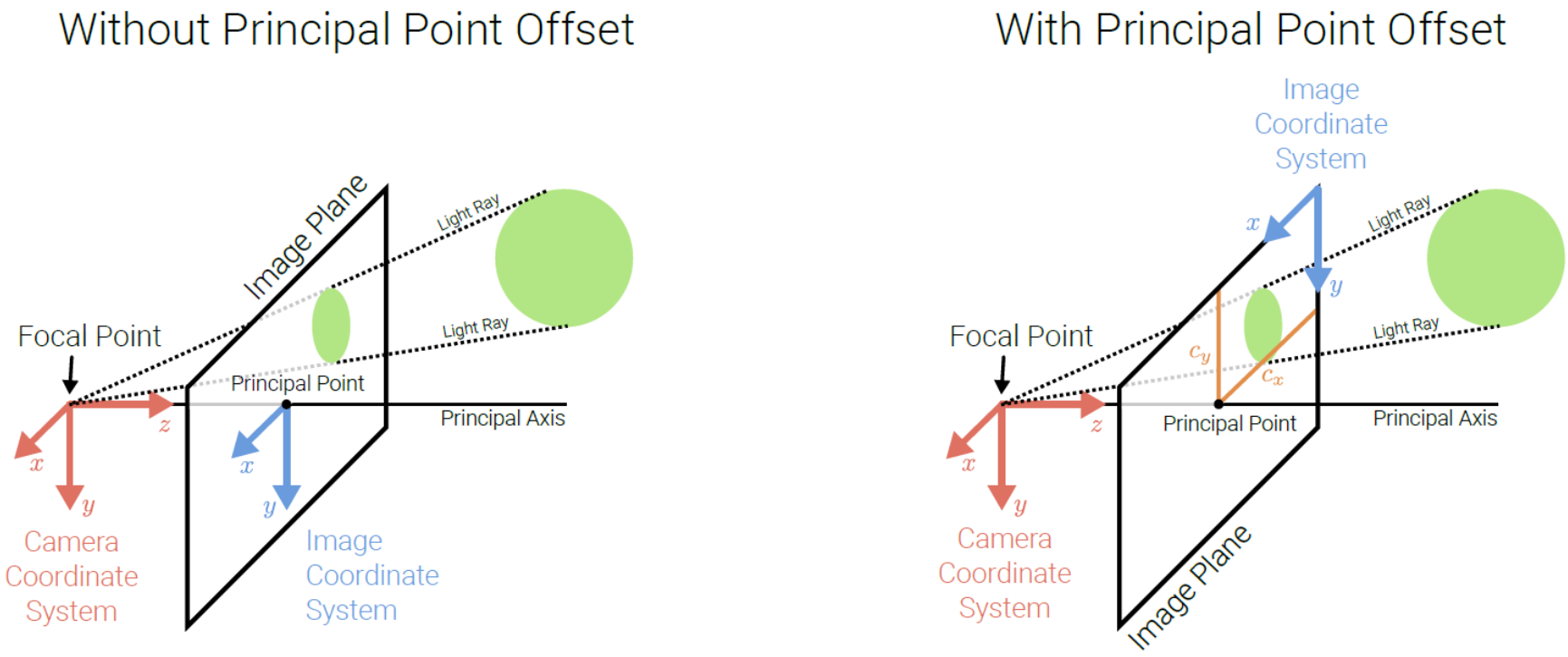
Perspective Projection

- 3D points in camera coordinates are mapped to the image plane by dividing them by their z component and multiplying with the focal length

$$\begin{pmatrix} x_s \\ y_s \end{pmatrix} = \begin{pmatrix} f x_c / z_c \\ f y_c / z_c \end{pmatrix} \quad \Leftrightarrow \quad \tilde{\mathbf{x}}_s = \begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \bar{\mathbf{x}}_c$$

- **This projection is linear when using homogeneous coordinates!**

Perspective Projection



- To ensure positive pixel coordinates, a **principal point offset** is usually added
- This moves the image coordinate system to the corner of the image plane

Perspective Project

- The **complete perspective projection model** is given by:

$$\begin{pmatrix} x_s \\ y_s \end{pmatrix} = \begin{pmatrix} f_x\, x_c/z_c + s\, y_c/z_c + c_x \\ f_y\, y_c/z_c + c_y \end{pmatrix} \quad \Leftrightarrow \quad \tilde{\mathbf{x}}_s = \begin{bmatrix} f_x & s & c_x & 0 \\ 0 & f_y & c_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \bar{\mathbf{x}}_c$$

  - The left 3×3 submatrix of the projection matrix is called **calibration matrix** $\mathbf{K}$
  - The parameters of $\mathbf{K}$ are called **camera intrinsics** (as opposed to extrinsic pose)
  - Here, f_x and f_y are independent, allowing for different pixel aspect ratios
  - The skew s arises due to the sensor not mounted perpendicular to the optical axis
  - In practice, we often set $f_x = f_y$ and $s = 0$, but model $\mathbf{c} = (c_x, c_y)^\top$

Perspective Project: Exercise

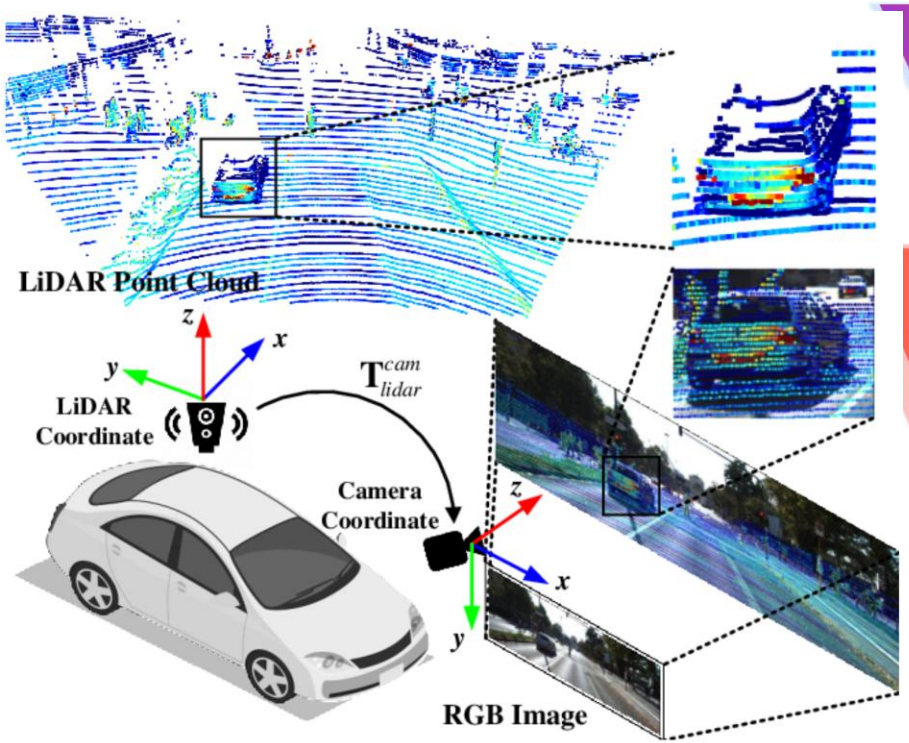
- Given a camera with calibration matrix $\mathbf{K} = \begin{bmatrix} 800 & 20 & 320 \\ 0 & 800 & 240 \\ 0 & 0 & 1 \end{bmatrix}$

- and a 3D point in the camera coordinate system $\bar{\mathbf{x}}_c = \begin{pmatrix} 10 \\ 5 \\ 2 \\ 1 \end{pmatrix}$

- Use the perspective projection formula to compute the 2D pixel coordinates (x_s, y_s) of the 3D point.

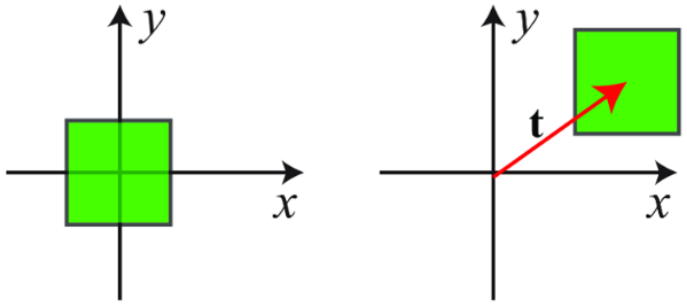
Coordinate Transformation

- **World Coordinate System (Global Frame)**: Used for global localization, path planning, and map matching. Static, large-scale coverage.
- **Vehicle Coordinate System (Body Frame)**: Centered on the vehicle. Used for real-time motion control and nearby obstacle detection. Dynamic, vehicle-centric.
- **Sensor Coordinate System (Local Frame)**: Unique to each sensor. Aligns with the sensor's installation position and orientation.



Coordinate Transformation

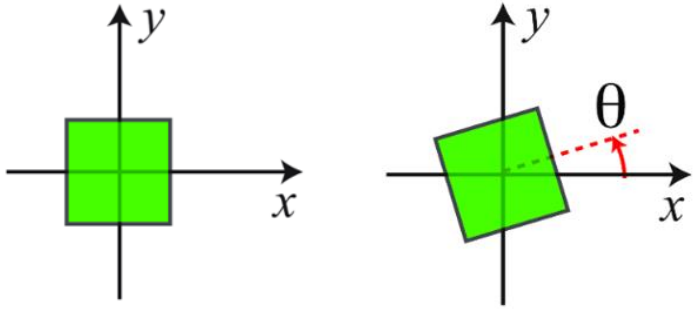
- 2D Translation



Translation

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- 2D Rotations



Rotation

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Coordinate Transformation

- 3D translation

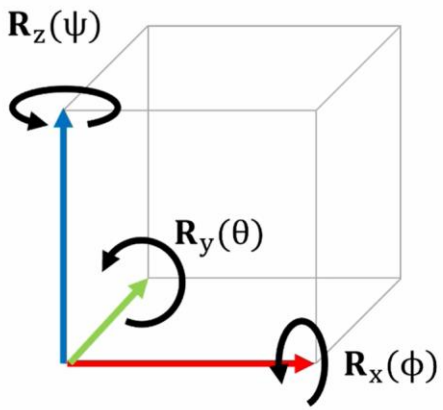
$$\mathbf{T} = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- 3D rotation

$$\mathbf{R}_x(\phi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\phi) & -\sin(\phi) \\ 0 & \sin(\phi) & \cos(\phi) \end{bmatrix}$$

$$\mathbf{R}_y(\theta) = \begin{bmatrix} \cos(\theta) & 0 & \sin(\theta) \\ 0 & 1 & 0 \\ -\sin(\theta) & 0 & \cos(\theta) \end{bmatrix}$$

$$\mathbf{R}_z(\psi) = \begin{bmatrix} \cos(\psi) & -\sin(\psi) & 0 \\ \sin(\psi) & \cos(\psi) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



Coordinate Transformation: Exercise

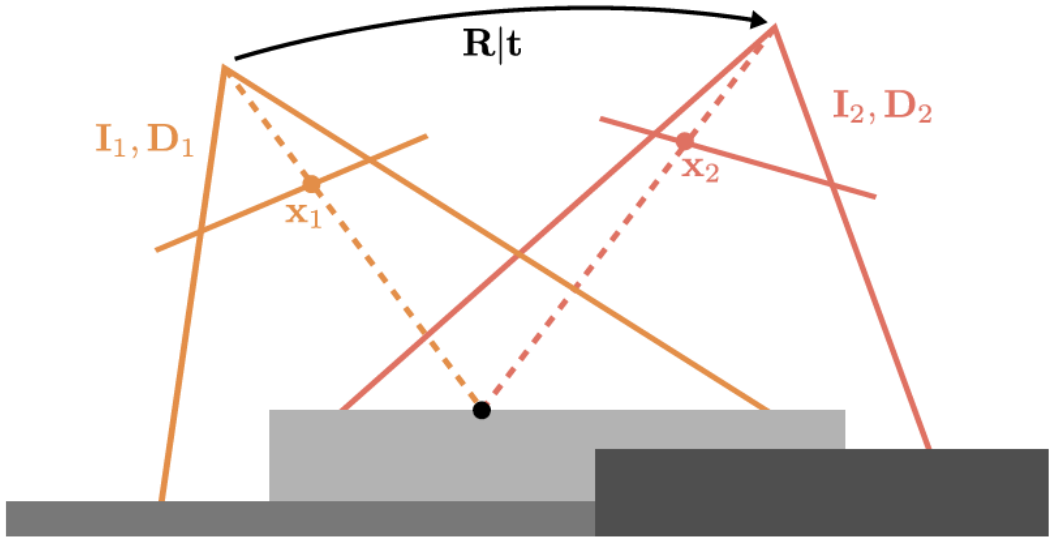
- Given a 3D point in homogeneous coordinates $\bar{\mathbf{x}}_3 = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 1 \end{pmatrix}$

- Apply the following sequence:
  - Rotate ϕ=90∘ around the **x-axis**.
  - Then rotate θ=90∘ around the **y-axis**.
  - Finally rotate ψ=90∘ around the **z-axis**.
  - Translate by t=(2,3,4)

- Compute the 3D point in resulting homogeneous coordinates

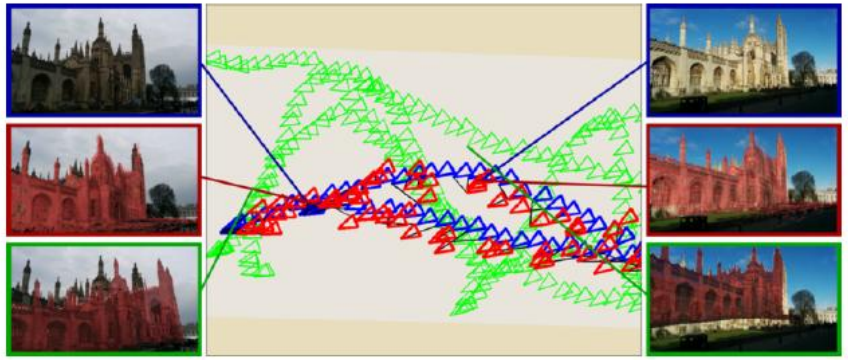
Direct Visual Odometry

- If we know per-pixel depth, we can **simulate** an image from a different view point



Learning-based Visual Localization

- **PoseNet**
  - Kendall, Grimes and Cipolla: PoseNet: A Convolutional Network for Real-Time 6-DOF Camera Relocalization. ICCV, 2015.
- Input: Single RGB image
- Output: 6 DOF camera pose p = (x, q)
  - x: 3D camera position
  - q: 3D camera rotation
- 23 Layer deep convolutional network based on GoogLeNet
- More robust than feature-based methods

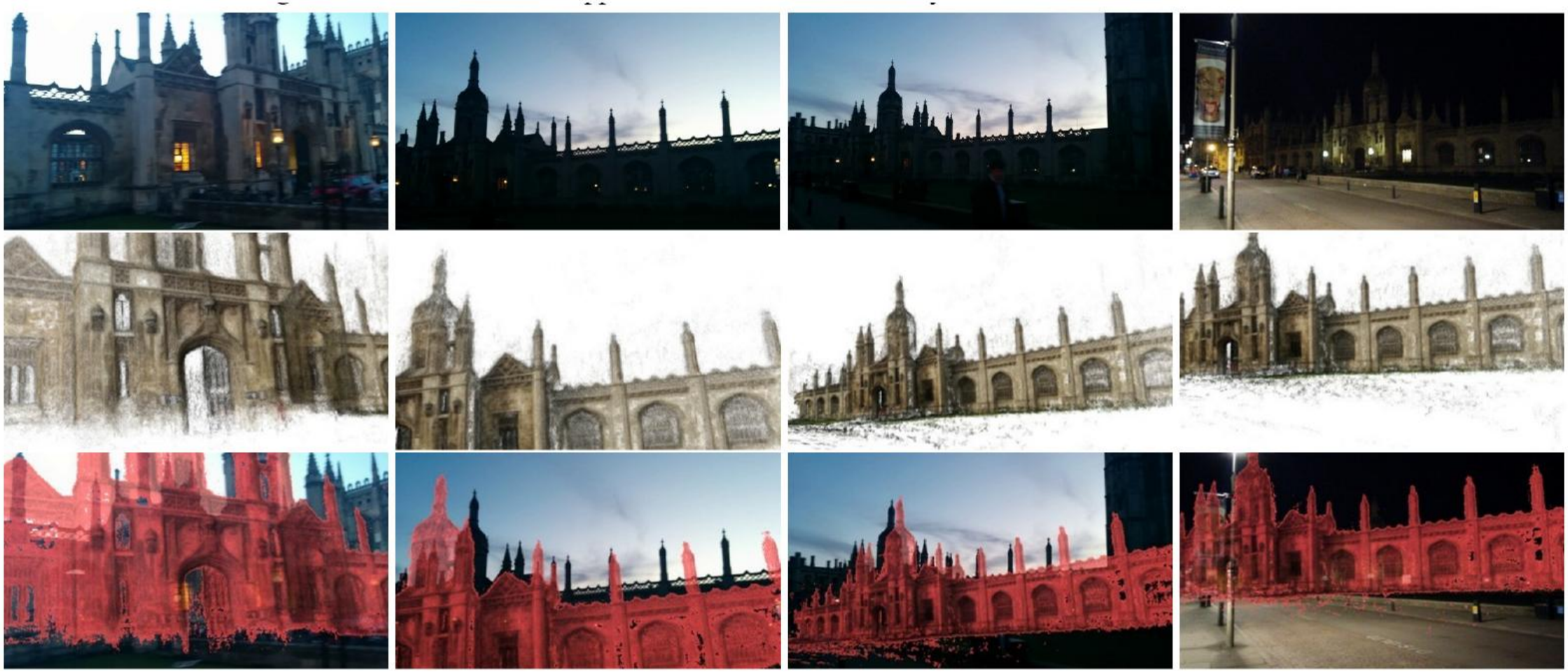


(green = train, red = pred, blue = GT)

Loss:

$$\mathcal{L} = \|\hat{\mathbf{x}} - \mathbf{x}\|_2 + \beta \left\| \hat{\mathbf{q}} - \frac{\mathbf{q}}{\|\mathbf{q}\|} \right\|_2$$

Learning-based Visual Localization



SLAM

- SLAM (Simultaneous Localization and Mapping) is a technology where vehicles, use on-board sensors to real-time estimate their pose and synchronously construct a dynamic environmental map containing lane lines, landmarks, obstacles, etc.
- Breaking the limitation of relying on pre-stored high-precision maps
  - Highway autonomous driving (blind spot compensation for sections without pre-stored maps)
  - L4-level urban road autonomous driving (dynamic environment update),
  - automatic parking (GPS-free scenarios in underground garages),
  - temporary navigation for areas with no map coverage in remote regions.
- This is a chicken-egg problem (localization requires mapping and vice-versa) ➔ Joint optimization of poses and map is necessary

SLAM: Core Modules

- **Front-End Perception and Odometry**: Relying on the multi-sensor fusion scheme of autonomous vehicles (LiDAR + Camera + IMU + GPS/BDS) for data collection.
- **Back-End Optimization**: To address cumulative errors and sensor noise caused by high-speed movement
- **Loop Closure Detection**: Identifies whether the vehicle has returned to a previously traveled section
- **Map Construction and Update**: Synchronously generates a semantic map suitable for autonomous driving, contains, e.g.,
  - environmental geometric information
  - marks lane attributes
  - traffic light states
  - positions of temporary obstacles (e.g., scattered objects, pedestrians)

Indirect SLAM Methods

MonoSLAM	PTAM	ORB-SLAM 2
+ monocular cameras	+ monocular cameras	+ monocular cameras + stereo cameras + RGB-D cameras
- no global consistency	- no global consistency	+ global consistency
Extended Kalman filtering of camera pose and point coordinates. Includes a motion model	Mapping as BA over keyframes, real-time tracking towards keyframe	Mapping as BA over keyframes, real-time tracking towards keyframe, loop-closing via place recognition
Filtering	Bundle adjustment	Bundle adjustment
- local accuracy -- global accuracy	+ local accuracy - global accuracy	+ local accuracy ++ global accuracy

Direct SLAM Methods

DVO-SLAM	LSD-SLAM	DSO
+ RGB-D cameras	+ monocular cameras + stereo cameras	+ monocular cameras + stereo cameras
+ global consistency	+ global consistency	- no global consistency
camera pose tracking towards keyframe	camera pose tracking towards keyframe	camera pose tracking towards keyframe, camera pose optimization in local keyframe window
+ depth from sensor	+ depth from stereo comparisons & filtering	++ depth optimization in local keyframe window
tracking-only & pose graph optimization	tracking-and-mapping & pose graph optimization	tracking-and-mapping & direct sparse bundle adjustment in local keyframe window with marginalization
+ local accuracy	+ local accuracy	++ local accuracy